Project - Low Level Design

on

Retail Rag

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Tamanna Katariya	EN22CS3011013
2.	Tanishka Patidar	EN22CS3011018
3.	Vivek Dhawle	EN22CS3011097
4.	Yogesh Ramani	EN22CS3011122

Group Name: 08 D9-D10

Project Number: GAI-01

Industry Mentor Name: Mr. Rishabh Tripathi

University Mentor Name: Prof. Divya Kumrawat

Academic Year: 2025 - 2026

Table of Contents

1. Introduction :

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design :

2.1. Application Design

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. References

1. Introduction

1.1 Scope of the Document

This document provides a comprehensive Low-Level Design (LLD) for the **Chat with Retail Documents (RAG System)**, a Generative AI-based system that enables users to interact with uploaded retail documents using intelligent question-answering.

The scope includes:

- Detailed system architecture and design
- Internal components and responsibilities
- Data flow and query processing pipeline
- API specifications
- Integration with LLM and vector database
- Performance and scalability considerations

This document acts as a technical reference for development and evaluation.

1.2 Intended Audience

This system is designed for:

- Retail analysts who want insights from reports
- Business users analyzing documents quickly
- Students working on AI-based document systems
- Developers building RAG applications

1.3 System Overview

The system allows users to:

- Upload retail documents (PDFs, manuals)
- Convert them into embeddings
- Store in a vector database
- Ask questions based on those documents

Core Features:

- Context-aware Q&A
- Document-based answers (no hallucination)
- Fast semantic search

Technologies Used:

- Python (FastAPI)
- Vector DB (FAISS / ChromaDB)
- LLM API
- Embedding model

2. System Design

2.1 Application Design

The system follows a layered architecture:

1. Presentation Layer

- Web interface (React / HTML)
- Upload + Chat UI

2. API Layer

- Handles routing and authentication

3. Business Logic Layer

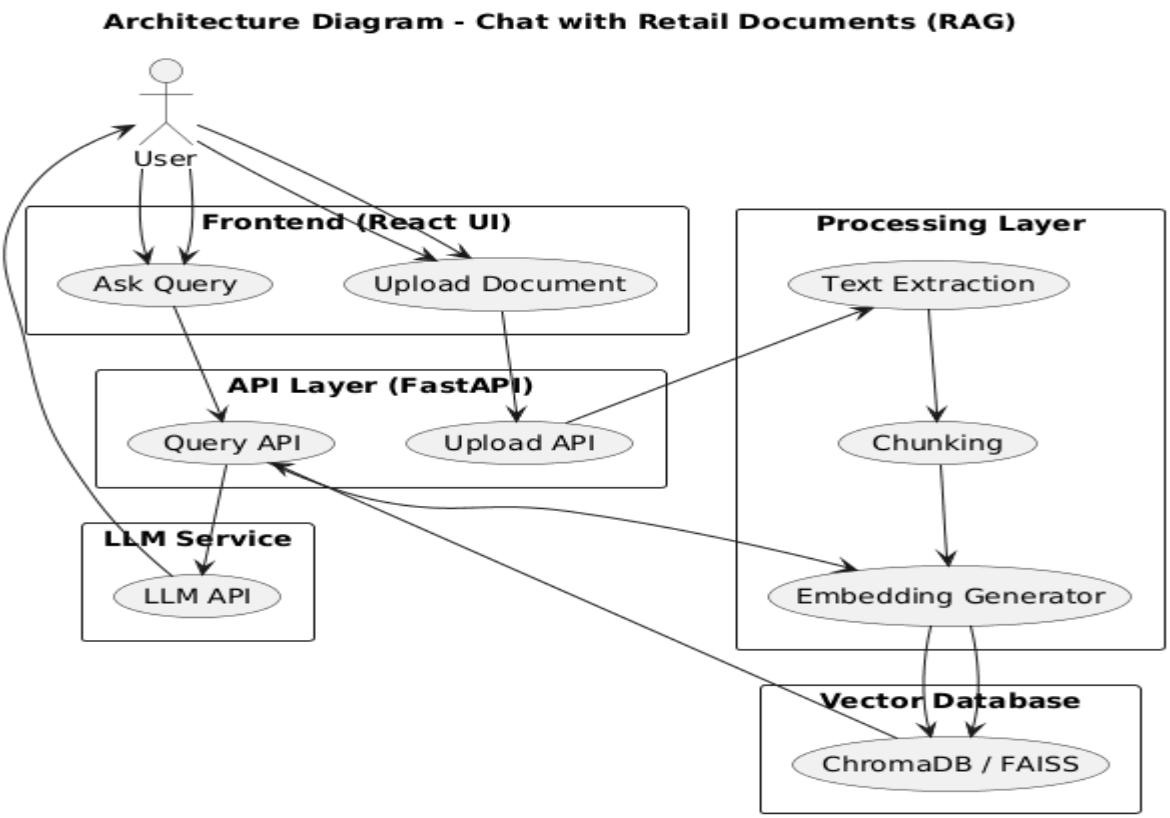
- FastAPI backend
- Handles document processing and queries

4. AI Layer

- LLM API (for answers)
- Embedding model

5. Data Layer

- Vector database storing embeddings
  -



2.2 Process Flow

Document Upload Flow

1. User uploads document
2. Backend extracts text

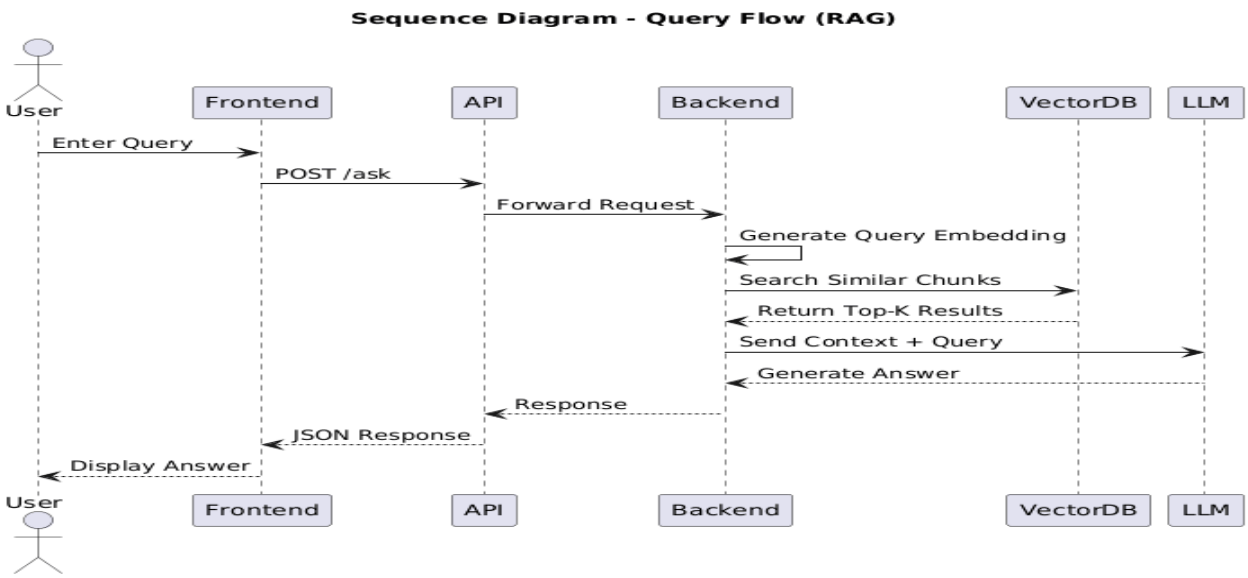
3. Text is chunked

4. Embeddings generated

5.Stored in vector DB

Query Flow

1. User asks question
2. Query converted to embedding
3. Vector DB retrieves relevant chunks
4. Context passed to LLM
5. LLM generates answer
6. Response shown to user



2.3 Information Flow

Input:

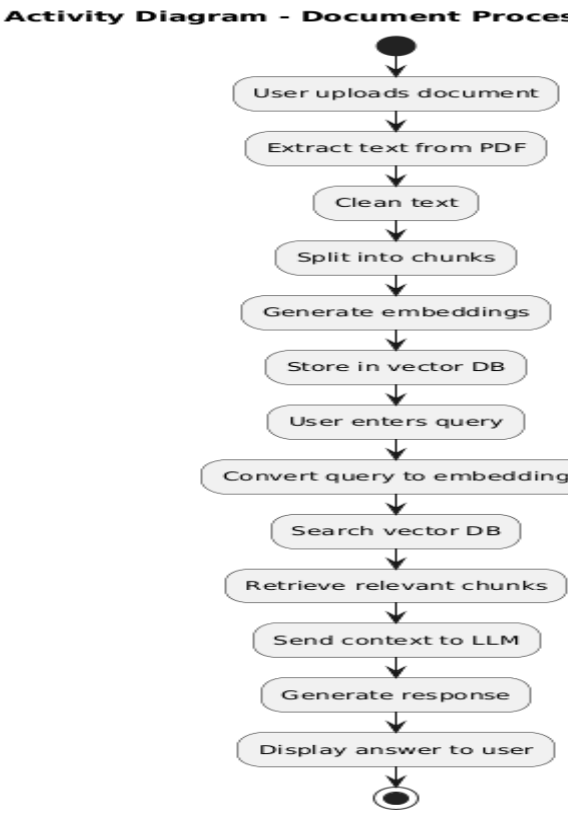
- PDF / document
- User query

Processing:

- Text extraction
- Chunking
- Embedding generation
- Similarity search

Output:

- Context-aware answer



2.4 Components Design

1. Frontend

- Upload documents
- Chat interface

2. Backend (FastAPI)

Modules:

Document Processor

- Extract text from PDF
- Clean data

Chunking Module

- plits text into chunks (300–500 words)

Embedding Module

- Converts text → vectors

Vector Store

- Stores embeddings
- Retrieves relevant chunks

LLM Service

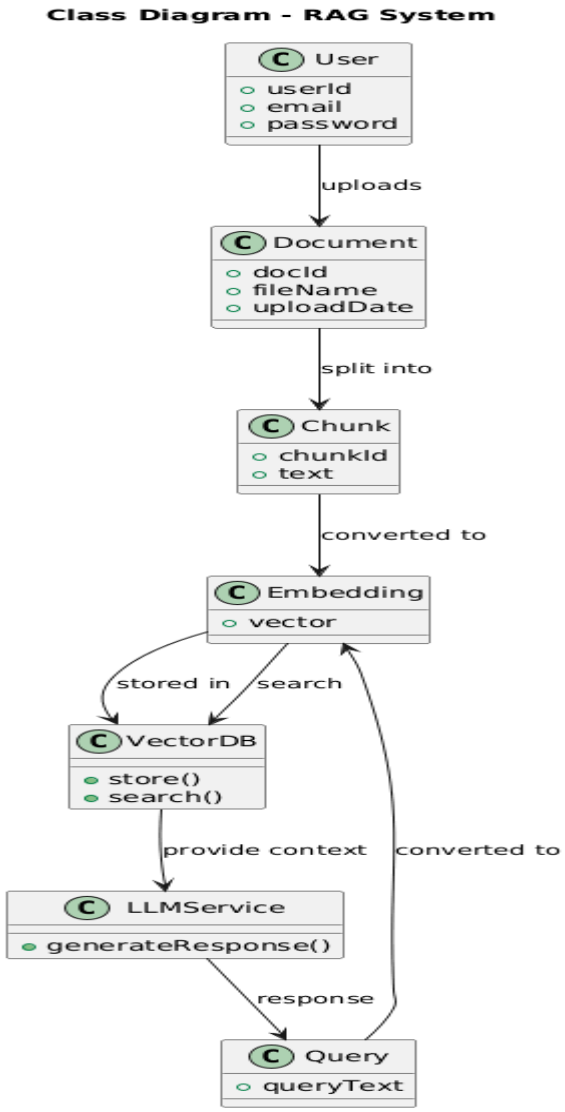
- Generates final answer

3. External Services

- LLM API (OpenAI / OpenRouter)
- Embedding model

4. Database

- Vector DB (FAISS / ChromaDB)



2.5 Key Design Considerations :

1. Modularity

- Separate components for easy updates

2. Scalability

- an handle multiple documents

3. Performance

- Fast retrieval using vector search

4. Accuracy

- Answers based only on retrieved context

5. Security

- JWT authentication
- Secure file handling

6. Maintainability

- Clean API structure

2.6 API Catalogue

1. Upload Document

Endpoint: `/upload-doc`
Method: POST

2. Ask Question

Endpoint: `/ask`
Method: POST

Request:

{ "query": "What is revenue growth?"}

Response: {

"answer": "The revenue growth is..."

}

3. Login

Endpoint: `/login`
Method: POST

4. Signup

Endpoint: `/signup`
Method: POST

5. Health Check

Endpoint: `/health`
Method: GET

3. References

- FastAPI Documentation
- FAISS / ChromaDB Docs
- OpenAI / LLM API Docs
- HuggingFace Embeddings